

IMPERIAL

Machine Learning for Design Engineers

Lecture 3: Learning Algorithms and Architectures

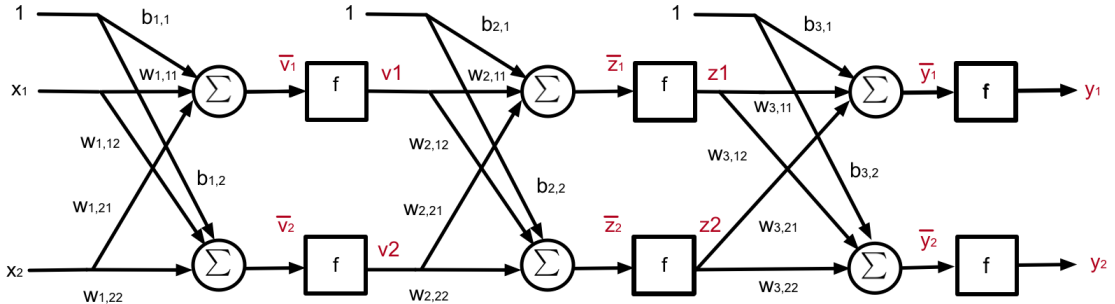
Pietro Ferraro

Table of Content

1. Backpropagation and AD
2. Activation Functions
3. Architectures

Backpropagation Recap

- Fundamental algorithm for training neural networks.
- Computes gradients of loss function w.r.t weights.
- Uses chain rule to propagate errors backward.



Limitations of Manual Differentiation

The reason for Backpropagation (BP) is that the alternatives (i.e., manual and symbolic differentiation) have the following drawbacks:

- Tedious for complex networks.
- Prone to errors in derivative calculations.
- Not scalable for large models.

Automatic Differentiation

As we mentioned last time BP is a specific case of a more general topic, called automatic differentiation:

- Computes derivatives automatically.
- Utilizes computational graphs.
- Enables efficient gradient computation.

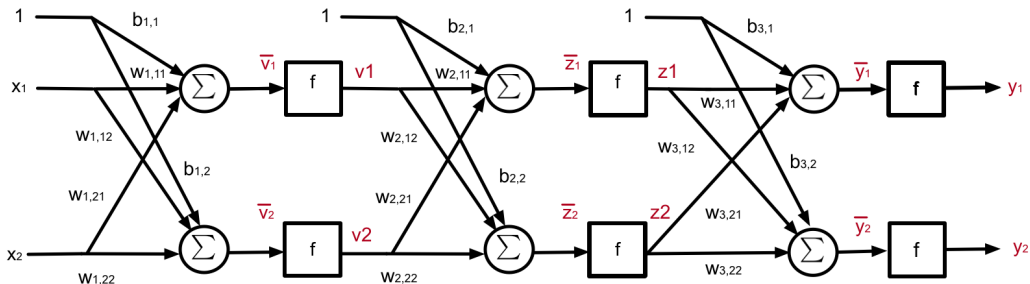
For example, consider the function:

$$f(x, y) = e^{-\sqrt{x^2+y^2}}$$

Computational Graph

Automatic differentiation works by building a computational graph:

- Nodes represent operations or variables.
- Edges represent dependencies.
- Facilitates derivative computation.



Step-by-Step Example

Autograd

Autograd is a tool that is used to compute the gradients automatically and quickly in two steps:

- **Forward pass:** for each node in the circuit compute the intermediate value for the specific input.
- **Backward pass:** given loss L , compute $\frac{\partial L}{\partial w_{ijk}}$, apply chain rule and, using the intermediate values obtained in the Forward pass, compute the gradients for each variable.

This makes the computation of gradients extremely quick and computationally efficient.

This is implemented automatically in Pytorch (but also in TensorFlow and JAX), so you won't ever have to calculate them manually.

Summary of how to train a network

The steps to train a network are then as follows:

- 1: Set up the network with a certain number of neurons and layers and a loss function $J(W)$. Divide your dataset into a "training set" and a "validation set".
- 2: Randomly initialise the weights and the biases of the network.
- 3: For each input x_i and output y_i perform forward pass and compute the loss $J(W)$.
- 4: Execute the backward pass (i.e., apply backpropagation) to compute the gradients $\frac{\partial J}{\partial w_{ijk}}$.
- 5: Apply gradient descent (or RMSE or Adam).
- 6: Repeat from point 3 until the desired loss is achieved.

Points 3 to 5 are usually called a training epoch. The desired level of loss is normally called the stopping condition of the training.

Activation Functions

Activation Functions

The sigmoid has been historically the most popular activation function, however it is used rarely nowadays

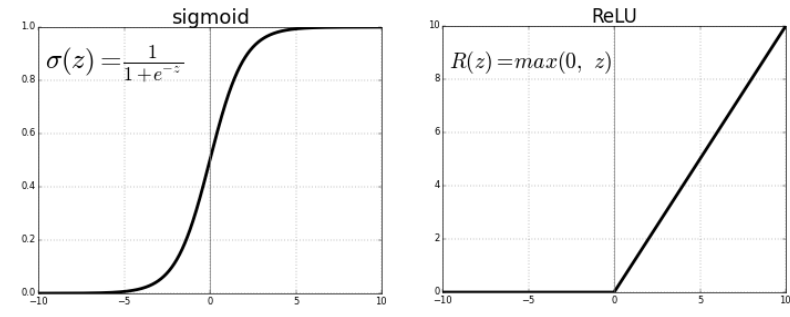


Figure: Source

Problems with Sigmoid

There are two main problems with the sigmoid:

- **Vanishing Gradients:** Derivatives approach zero for large $|x|$.
- **Slow Convergence:** Saturation regions hinder learning.

In fact:

- Derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.
- For large $|x|$, $\sigma(x) \approx 0$ or 1 .
- Therefore, $\sigma'(x) \approx 0$.

This means that deep layers will not train. Can anybody explain why?

ReLU Activation Function

ReLU on the other hand has the following properties:

- Definition: $\text{ReLU}(x) = \max(0, x)$.
- Output range: $[0, \infty)$.
- Computationally efficient.

Which lead to

- Mitigates vanishing gradient problem.
- Sparse activation (efficient computation).
- Faster convergence in deep networks.

Potential Issues with ReLU

ReLU however are not perfect as they still have some issues:

- **Dying ReLU:** Neurons can get stuck in the zero gradient region.
- Remedies include Leaky ReLU, ELU and random resets of the weights.

Leaky ReLU:

- Definition: $\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases}$
- Allows a small, non-zero gradient when $x < 0$.

Choosing Activation Functions

In general for shallow networks (with not many hidden layers) sigmoids work fine and they do not cause issues. For deep networks ReLU is by far the most reasonable choice (however there are hundreds of papers in literature about different activation function or combination of activation functions).

Let's now take a look at the following architectures:

- ANN for regression
- ANN for classification

Architectures

Universal Approximation Theorem

The reason ANNs are so powerful and they are able to achieve the results that we have seen over the years is mostly due to the fact that ANNs are Universal Approximators:

- Neural networks can approximate any continuous function to an arbitrary degree of accuracy.
- Requires sufficient neurons and layers.
- Activation functions play a key role.

Universal Approximation Theorem

A classical result from Cybenko, in his seminal paper from 1989 is that given a continuous function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, (you will often see this written as $f \in C^0(\mathbb{R}^d)$), for a given accuracy ϵ_n , there exists a number of neurons n , such that:

$$NN(x) = \sum_{i=1}^n v_i h_i(w^T x + b)$$

$$||f - NN||_{\infty} \leq \epsilon_n$$

Activation Functions for Regression

The previous result means that ANNs are excellent to solve non linear regression problems, in which our aim is to capture the underlying relationship between the input and the output of our dataset. In other words we want to use a neural network $\text{NN}(\cdot)$ to approximate a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. This can be done as follows:

- Linear activation in output layer.
- Measures difference between predicted and true values.
- Regression tasks are (usually) solved using the MSE loss function,
$$J = \frac{1}{n} \sum_{i=1}^n (\text{NN}(x_i) - y_i)^2.$$

Overview of Classifiers

While for a regression problem our aim is to approximate a continuous function, in a classification problem we want instead to approximate a function $f : \mathbb{R}^d \rightarrow D$, where D is a finite set.

Consider, for example

- A set of images of cats and dogs
- Being able to say whether a patient has diabetes or not
- Discriminating which language a certain text belongs to

A binary classifier

If the problem has only two classes to distinguish from, this can be easily solved using a binary classifier, which employs a sigmoid as output (why not the sign function?)

But what if we have k classes?

One-hot vectors

The first thing to do is to convert our output y into a one-hot vector z_i . A one-hot vector is a vector $z_i = [0, 0, \dots, \underbrace{1}_{\text{i-th position}}, \dots, 0]$. For example:

Softmax

Once all our outputs have been converted into one-hot vectors, we will train the following architecture:

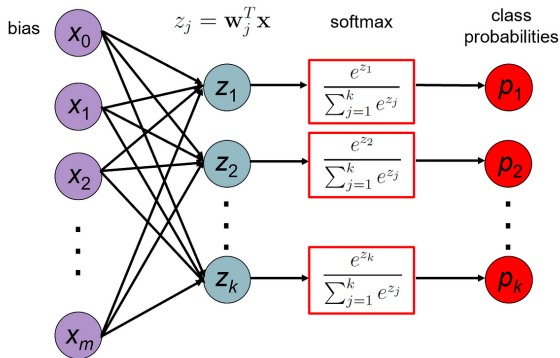


Figure: Source

Softmax and probability

Notice that The output of your classifier does not strictly indicate which class your input belongs to but rather provides a series of probabilities per each class.

Accordingly, we will use the following loss function for this kind of architectures:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

This loss function is called "Cross-entropy" and it has many advantages over MSE for classification problems:

- It encourages correct classification
- It pairs very well with softmax (more on this in the lab)

Training either a regression problem or a classifier

First step is to identify whether the problem you are working with is a regression problem or a classifier. Then:

- Choose appropriate architecture and loss functions.
- Split your dataset into a training set and into a validation set
- Apply forward pass and backward pass using the training set. Compute loss on both training set and validation set.
- Apply gradient descent to update the weights.
- Repeat the previous two steps until loss stops decreasing in the training set.

What if the loss goes very low in the training set but it is very high in the validation set?

Questions and Discussion

Thank you!

Any Questions?